Criteria for laboratory work

Each laboratory paper has two separate graded assignments:

- **Test** (10% of final grade)
- **Thesis defence** (5% of the final grade), which in turn includes:
  - Oral demonstration
  - Report - must be uploaded to moodle at least 24 hours before the defence

  We assess the demonstration according to the criteria below, but also taking into account the quality of the report - i.e. we may downgrade if the report does not meet the requirements.

  Please note:
  - All the steps are sequential, i.e. you can only get a 7 if you do a task for 5 points, etc.
  - If the quality of the reference is completely unsatisfactory, a negative assessment is possible, even if the demonstration was excellent.

Lab 1. Combinational logic circuits and their synthesis

Criteria for defence

All the steps are sequential, i.e. you can only get a 7 if you do a task for 5 points, etc.

Criterion	Maximum score	What you have to do (show/explain/change)
Function overwriting and minimisation, and diagramming of AND, OR, NOT elements, simulation	5	• Demonstrate a correct function, appropriate to the task, written in normal disjunctive form. • Be able to explain and demonstrate how this function is constructed. • Minimise the function (using a Carnot table or other techniques). • Be able to explain and show how the function was minimised. • Be able to answer questions from lecturers about function generation, minimisation. • Demonstrate a minimised function-compliant (and correct) scheme consisting of AND, OR, NOT elements. • Demonstrate a simulation of the scheme, and be able to explain how (and if) the scheme is shown to work correctly. • Be able to answer questions from lecturers about the scheme and its simulation. • Be able to make changes to the diagram.
Schematic diagramming of NAND (or NOR) elements, simulation	7	• To demonstrate the function transformed according to De Morgan's law. • Be able to define and explain De Morgan's Law. • Demonstrate a reordered function-compliant (and correctly functioning) scheme consisting of NAND (or NOR) elements. • Demonstrate a simulation of the scheme, and be able to explain how (and if) the scheme is shown to work correctly. • Be able to answer questions from lecturers about the scheme and its simulation. • Be able to make changes to the scheme.
Multiplexer (MUX) circuit design, simulation	10	• Demonstrate the transformed function by raising two (or more) address variables. • If the transformed function has been minimised, be able to explain the minimisation process. • Be able to explain how address variables are chosen • Be able to explain the operation of a multiplexer • Demonstrate a multiplexer scheme that corresponds to the transformed function (and works correctly). • Demonstrate a simulation of the scheme, and be able to explain how (and if) the scheme is shown to work correctly. • Be able to answer questions from lecturers about the scheme and its simulation. • Be able to make changes to the scheme.

Laboratory reports - Evaluation criteria

Criterion	What you have to do

Report structure	• Parts of the report include
	  ○ Cover
	  ○ Introduction - a brief description of the job in your own words (do not copy text from descriptions/slides)
	  ○ Task
	  ○ Workflow description
	  ○ Conclusions

General reporting procedure and language	• Comply with the general KTU requirements for the processing of thesis reports.
	• Write in correct, academic Lithuanian.
	• Neatly number tables and figures.
	• Present logical and other mathematical expressions in a neat, standard format using the functions provided by text editors (e.g. Word -> Insert Equation).
	  ○ E.g.: F= $a\overline{b}\overline{c} + \overline{a}bc + abc$
	  ○ Please note - $\overline{a}\overline{b}\overline{c} \neq \overline{abc}$
	• Do not include text, tables and formulae in the form of pictures or photographs.

Presenting and commenting on work progress and results	• Submit all tables, charts and diagrams produced during the work • Present the simulation results for all schemes. • Comment on each table/figure with text, explaining what was done, how it was done, what was obtained and how to interpret the results. Remember - if the tables/figures are not described/explained in some way in the text, they are meaningless. • In the simulation images provided, mark the relevant points, explaining what and why the simulated scheme is working correctly (or incorrectly). • If you have worked with FPGA (Works 3,4) - provide pictures of how they work.
Drawing conclusions	• Conclusions should be in bullet points. • Provide your observations and insights, not just a statement of fact (done... done... etc.) • Write what worked, what didn't, what you would do differently next time, what you would recommend to others. • Comment on the results, explaining how you see them as true/false.

Boolean functions and combinational logic schemes

Content

Criteria for laboratory work **1**

Boolean functions and combinational logic schemes **3**

1. Boolean functions 3

1.1 Displaying Boolean functions 3

1.2 Minimising Boolean functions 5

2. Combinational logic circuits 8

2.1 Concept of a Combination Scheme 8

2.2 Implementation of the scheme using AND, OR, NOT elements 9

2.3 Implementation of the scheme using NAND elements 9

2.4 Creating a scalar expression 11

2.5 Implementation of the scheme using multiplexers 14

1. Boolean functions

Boolean algebra is widely used to describe the functioning of digital circuits. It is an algebra (***B***, •,∪ , ¬, **0**, **1**) consisting of a finite set **B of** elements, whose elements can take the values 0 or 1, and three defined operations on the elements:

- **An AND** operation (logical multiplication or **conjunction**), denoted by •, & or ^; if it is clear from the context that it is a conjunction of variables, this operation sign is often omitted;
- **OR** an operation (logical addition or **disjunction**) denoted by +, v or ∪ ;
- **NOT** operation (negation or **inversion**), indicated by a ¬ or a dash above the variable (expression).

Thus, each logic circuit implements a logic (Boolean) function (output y-value) whose arguments are Boolean variables (input x-values).

1.1 Displaying Boolean functions

Boolean functions are represented by:

- correctness tables, giving a logical value for each possible combination of values of ***the n*** variables (number of combinations equals 2^n);
- analytical expressions, where functions are expressed in formulae;
- numerical form, where the combinations of variable values for which the function takes the value 1 are replaced by the decimal equivalents of the binary codes (also known as cubes) of the combinations;
- Karnaugh (Weich) maps, where Boolean functions are represented graphically.

Example. Let us construct a three-variable function $y = f(x_1, x_2, x_3)$, which takes on the value of one only when an odd number of inputs takes on the value 1 (Table 1).

Table 1. Correctness table for the function $y = f(x_1, x_2, x_3)$

x_1	x_2	x_3	y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

To reduce the size of the table, it often only includes combinations of variable values where the function is 1. This is illustrated in Table 2.

Table 2. Abbreviated validity table for the function $y = f(x_1, x_2, x_3)$

x_1	x_2	x_3	y
0	0	1	1
0	1	0	1
1	0	0	1
1	1	1	1

Using the correctness table, let's write down the analytical expression of the given function - the formula. In the simplest case, the function can be written as a disjunction of the conjunctions corresponding to those values of the variables with which the function is equal to one:

$$y = f(x_1, x_2, x_3) = \overline{x_1}\,\overline{x_2}\,x_3 + \overline{x_1}\,x_2\,\overline{x_3} + x_1\,\overline{x_2}\,\overline{x_3} + x_1\,x_2\,x_3 .$$

Any conjunction that includes all the letters ($\overline{a}$ variable called **a letter** or its inverse) is called a **minterm**.

The resulting function can be further simplified by putting common minterm members before the parentheses:

$$y = f(x_1, x, x_{23}) = \overline{x_1}\,(\overline{x_2}\,x_3 + x_2\,\overline{x_3}) + x_1\,(\overline{x_2}\,\overline{x_3} + x_2 x_3),$$

or, using the rules of Boolean algebra, replace it with another expression:

$$y = f(x_1, x_2, x_3) = \overline{x_1}\,(x_2 \oplus x_3) + x_1\,\overline{(x_2 \oplus x_3)} = x_1 \oplus x_2 \oplus x_3 .$$

When writing the function in numerical form, the numbers of the minterms (or the decimal equivalents of the binary codes of the variable combinations) for which $y = 1$ are given (see the left-hand column of the table); for the function in question we have:

$$y = f(x_1, x_2, x_3) = (1, 2, 4, 7) .$$

For Boolean functions with up to 6 variables, it is proposed to use **Karnaugh maps**. They are constructed by transcribing the results of the correctness table into a two-dimensional table with rows and columns numbered in Grey's code. Each cell of the table corresponds to one combination of inputs.

The Karnaugh map of the function $y = f(x_1, x_2, x_3) = (1, 2, 4, 7)$ is shown in Table 3 (the unit values of the function have been written in the appropriate cells using the correctness table).

Table 3. Karnaugh map of the function $y = f(x_1, x_2, x_3) = (1, 2, 4, 7)$

x_1x_2 \ x_3	0	1
00		**1**
01	**1**	
11		**1**
10	**1**	

1.2 Minimising Boolean functions

The same Boolean function can usually be represented by several expressions. Usually the Boolean function is minimised, that is, an analytical expression of the function is obtained with fewer operations, conjunctions or conjunction letters. This allows the scheme to be implemented with fewer logical elements.

For minimising small functions, the so-called trial-and-error approach works well, where the minimisation is based on the designer's knowledge and experience. The following procedure is used for minimisation:

- Using the axioms and theorems of Boolean algebra, the extended disjunctive normal form can be rearranged to obtain a simpler disjunctive normal form with fewer conjunctions or fewer letters in the conjunctions;
- by grouping conjunctions and putting common letters before parentheses, a shorter function expression can be obtained;
- By using more complex operations (modulo 2, etc.) in addition to the basic operations (disjunction, conjunction and inversion), the function expression can also be shortened.

To minimise functions with up to 6 variables, it is proposed to use **Karnaugh map**. This approach makes effective use of the designer's ability to visually identify the characteristic arrangement of units in the diagram. Optimal groups of '1' or '0' corresponding to shorter conjunctions are found. These conjunctions are used to write the minimum Boolean expression.

Example. We have a function $\boldsymbol{y = f(a, b, c, d)}$, whose abbreviated correctness table is shown in Table 4.

Table 4. Correctness table for the function $y = f(a, b, c, d)$

a	**b**	**c**	**d**	***y***
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1
1	1	1	1	1

Using the fairness table, we create a Karnaugh map . As this is a function of 4 variables, we have 16 different combinations of values, so the full truth table has 16 rows and the Karnaugh map has 16 cells. For this reason, the diagram is represented by a 4 × 4 table. The values of the rows and columns are denoted by a Grey code. The Grey code ensures that adjacent cells differ by the value of only one variable. If the function takes the value '1' for a particular combination of variable values, a 1 is entered in the corresponding cell of the Karnaugh map.

A Karnaugh map seeks a simplified way of expressing the information written down. Adjacent units are grouped in the table according to certain rules:

- The groups of minterms in a Karnaugh map must be rectangular. The number of the group "1" can only be a power of two (2,4,8...).
- Groups of minterms should be as large as possible.
- Groups may overlap each other to make each group as large as possible.
- The table is toroidally connected. This means that the rectangles of the groups can cross the edges of the table. The left edge cells are adjacent to the right edge cells, the top cells to the bottom cells. For example, all four corners of the table are considered adjacent.

Let's group the elements of the Carnot table into groups (see Table 5). The optimal groups in this table are highlighted by different coloured frames. In some places they overlap. The group defined by the red line (red) consists of a rectangle of size 4 × 1, the group defined by the blue line (blue) consists of a rectangle of size 1 × 4, and the group defined by the green line (green) consists of a rectangle of size 2 × 2 (because the edges of the table are adjacent).

Table 5. Karnaugh map of the function $y = f(a, b, c, d)$

cd \ ab	00	01	11	10
00		1	1	
01		1		
11	1	1	1	1
10		1	1	

The conjunction corresponding to the group is determined by analysing the column and/or row codes of the group. For example, in the green group:

- the column codes are 01 and 11; in the first column the variable $\boldsymbol{a}$ = 0 and in the second column $\boldsymbol{a}$ = 1, so it is not included in the conjunction; while the variable $\boldsymbol{b}$ is 1 in both columns, so $\boldsymbol{b}$ is included in the conjunction corresponding to the green group;
- the row codes are 00 and 10; hence variable $\boldsymbol{d}$ is the same in both rows and equals 0, so we include $\overline{\boldsymbol{d}}$ in the conjunction corresponding to the green group; variable $\boldsymbol{c}$ takes different values in the rows of the group, so we exclude it from the conjunction.

Thus, the green group corresponds to conjunction $\boldsymbol{b}\,\overline{\boldsymbol{d}}$.

Analysis of the other groups gives the following results:

- the red group corresponds to the conjunction $\overline{\mathbf{a}}\,\mathbf{b}$;
- the blue group corresponds to conjunction **c d**.

So, using the Karnaugh map , we minimised the function and got:

$$y = b\overline{d}+\overline{a}b+cd.$$

This minimised expression can be obtained without using a Karnaugh map , but by applying the axioms of Boolean algebra, but it becomes rather complicated if the number of minterms is large.

The resulting function can also be written in a different way by putting $\boldsymbol{b}$ before the brackets:

$$y = b(\overline{d}+\overline{a})+\mathbf{cd}.$$

In the case of six variables, the Karnaugh map is much larger, so the minimisation rules are more complex. Taking into account the properties of the Grey code, let us draw the main (red dashed) and additional (blue dashed) axes of symmetry in the Karnaugh map (see Table 6).

Table 6. Six-variable Karnaugh map

def \ abc	000	001	011	010	110	111	101	100
000								
001								
011								
010								
110								
111								
101								
100								

In fact, let's look at the order of the **abc** codes in the top row. The first column of codes to the left of the red dashed line is 0, the first column of codes to the right is 1. If the first columns are discarded, the following code sequence remains: 00 01 11 10 | 10 11 01 00. We can see that the codes on the right are like a mirror image of the codes to the left of the vertical dash.

The situation is similar with the positioning of the **def** codes in the vertical column. Thus, the red dotted lines are actually the axes of symmetry for the whole diagram.

The blue dashed lines are the axes of symmetry within each of the quarters into which the red dashed lines divide the diagram.

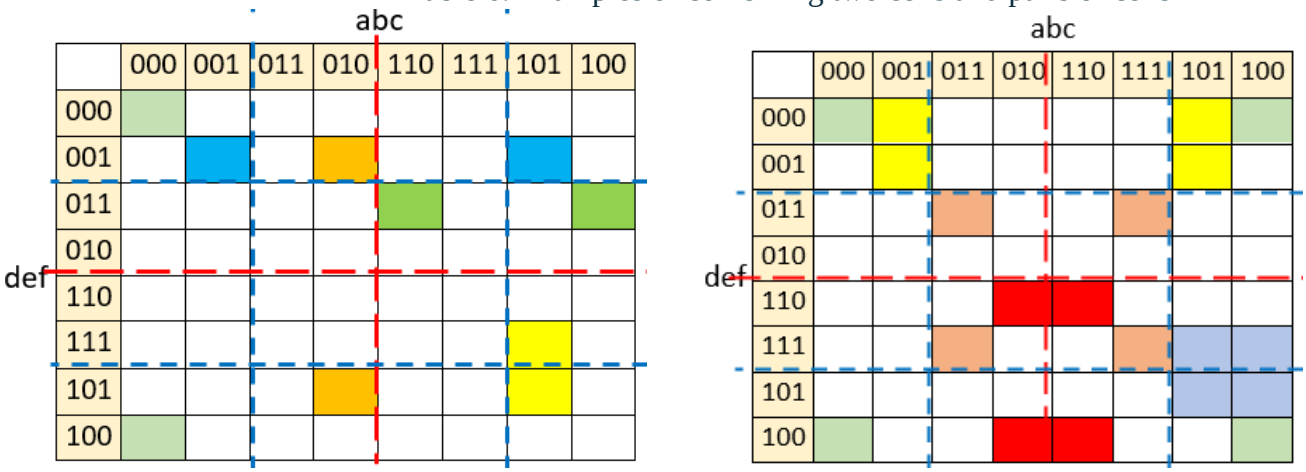
Taking these axes into account, the following rules for combining the two cells can be formulated:

- any adjacent cells can be merged,
- any cells that are symmetrical to the main and secondary axes of symmetry can be combined.

The rules for combining pairs of cells sound similar, but in this case there are additional possibilities.

In the diagrams below, the cells that satisfy the rules for combining two cells and pairs of cells are coloured the same colour:

Table 6. Examples of combining two cells and pairs of cells



2. Combinational logic circuits

Logic circuits are divided into **combinational** circuits and **sequential** circuits (also known as memory circuits, sequential circuits in the literature).

2.1 Concept of a Combination Scheme

Logic circuits consisting of interconnected logic elements, without loops (feedback) and memory elements, in which the values of the vector ***Y*** of the output signals are uniquely determined by the values of the vector ***X of the*** input signals present at the time, are called **combinational**. Combinational circuits are often referred to as logic circuits without feedback.

The combination scheme is summarised in Figure 1.

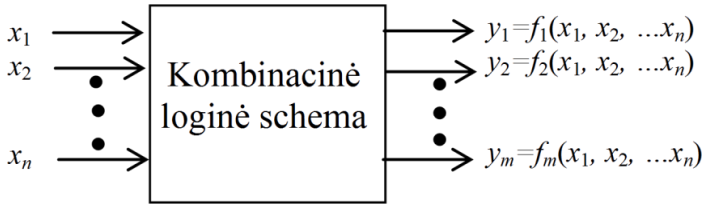


Figure 1: Combinational logic diagram

Combinational logic circuits are designed in the following order:

1. The logical functions to be implemented by the designed circuit and their arguments (logical variables) are defined.
2. Logical functions are written down; logical functions can be represented by truth tables or by expressions of the disjunctive form of logical functions . [1]
3. Minimising logic functions by one of the well-known methods (Karnaugh map s, Quine-McCloskey, etc.) yields minimal (or near-minimal) expressions for these logic functions. These expressions can be rearranged in such a way that they can be more easily realised by logic elements.
4. Synthesis: ready-made logic functions are implemented in the desired logic elements. A combinatorial logic diagram is obtained. Depending on the speed and power consumption requirements, special automated methods for the synthesis of combinational circuits can be used to obtain circuits with the optimum number of logic elements.

[1] The disjunctive form is the logical sum of conjunctions (disjunction).

5. Once the combinatorial scheme is synthesised, a control test is set up, which is a sequence of variable values fed to the inputs of the designed scheme to check whether the scheme implements the required functions.

2.2 Implementation of the scheme using AND, OR, NOT elements

The above expression for the function $y = f(x_1, x, x_{23}) = (1, 2, 4, 7)$ (as well as any expression written in normal disjunctive form) can be implemented using three types of elements: AND, OR and NOT. A combinatorial flowchart implementing the above function (Table 5) is shown in Figure 2.

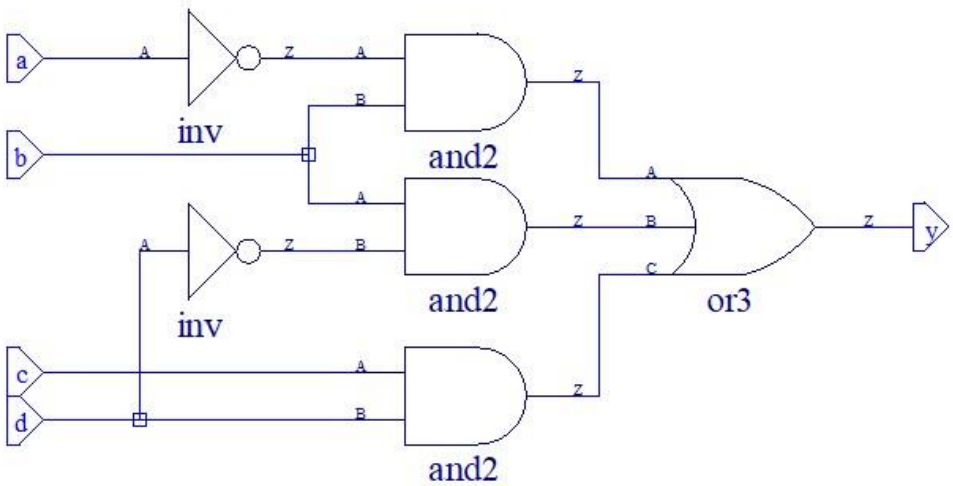


Figure 2: Logic diagram of the function in Table 5

This diagram was drawn with the Lattice Diamond graphical editor; the ***and*** elements implement the AND function (conjunction), the ***OR*** element the OR function (disjunction), and the ***inv*** elements the NOT function (inversion).

2.3 Implementation of the scheme using NAND elements

De Morgan's law allows any Boolean function to be expressed only in terms of negation and conjunction operations, i.e. in the minimal (also known as universal) element basis.

De Morgan's Law is written as follows:

$$a + b = \overline{\bar{a}\,\bar{b}} \text{ or } a\,b = \overline{\bar{a} + \bar{b}}$$

Applying De Morgan's law to the above expression, let's rearrange it by initially inverting each half of the expression twice (we know that $\bar{\bar{x}} = x$):

$$\bar{\bar{y}} = \overline{\overline{\overline{b(\bar{d}+\bar{a})}+cd}}.$$

Then

$$y = \overline{\overline{\overline{b(\bar{d}+\bar{a})}}\;\overline{cd}} = \overline{\overline{b\;\overline{\bar{d}\,\bar{a}}} - \overline{c\,d}}.$$

The restructured function is easy to implement using only NAND elements.

The combinational scheme obtained using NAND logic cells is shown in Figure 3.

We can see that the scheme in Figure 2 uses three types of elements, while the scheme in Figure 3 requires only one type of element.

It is quite difficult to transform a more complex expression in this way, because it produces "multi-storey" inversions. In contrast, there is a simple way to transform a scheme of AND, OR, NOT elements into a scheme of NAND elements and inverters.

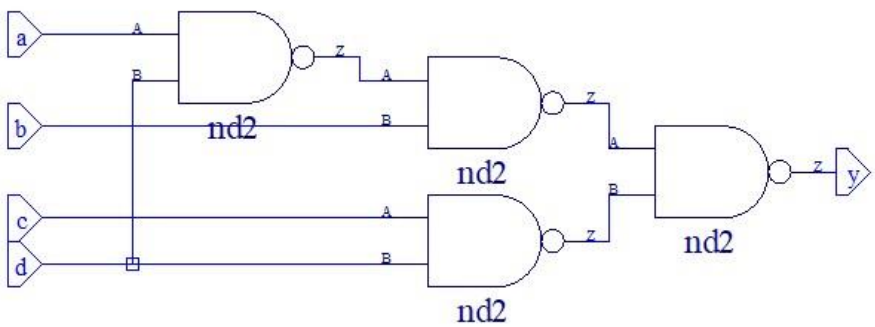


Figure 3: Combinational diagram obtained using NAND logic elements

A two-tier scheme, with the first tier containing the elements AND and the second tier containing the combining element OR, is equivalent to a scheme with the same configuration, where all the elements are NAND (see Figures 4(a) and (b)).

In fact, the diagram in Figure 4(a) of the AND, OR, NOT elements implements the function

$$y = a\bar{b} + bcd.$$

The function realised by the scheme in Figure 4(b) is

$$z = \overline{\overline{a\bar{b}}\ \ \overline{\overline{bc}\,\overline{d}}} = a\bar{b} + bcd.$$

Obviously, $y = z$. We can see that it was sufficient to replace each element AND and the element OR by the elements NAND and the configuration of the diagram remained the same.

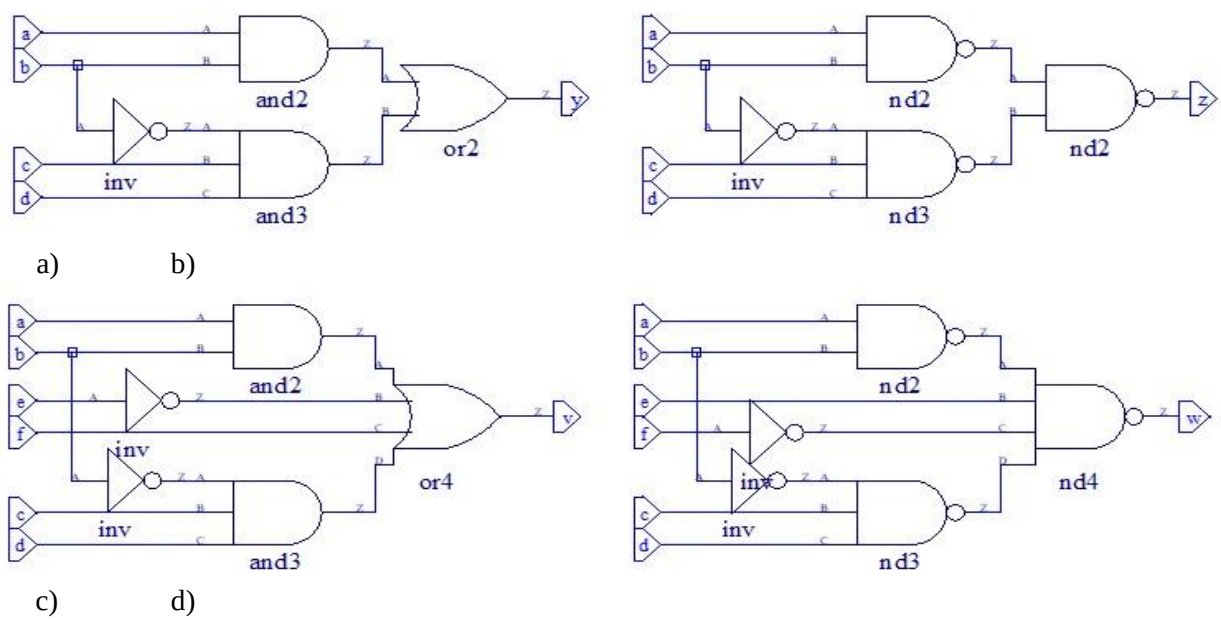


Figure 4: Comparison of function realisations with AND, OR, NOT and NAND elements

However, if variables are fed into the combining element OR, the variables must be inverted into the corresponding combining element NAND.

In Figure 4(c), the function $v = ab + \bar{b}cd + \bar{e} + f$ is realised by the scheme. The function realised by the scheme in Figure 4(d) is

$$w = \overline{\overline{a\bar{b}}\ \ \overline{\overline{bc}\,\overline{d}}\ \ \overline{e}\ \ \overline{f}} = \overline{\overline{ab}} + \overline{\overline{\overline{bc}\,\overline{d}}} + \overline{\bar{e}} + \overline{\bar{f}} = ab + \bar{b}cd + \bar{e} + f.$$

We see that $\boldsymbol{v} = \boldsymbol{w}$.

Note that in (c) the OR combining element (or4 in the diagram) is fed with the variables $\overline{\mathbf{e}}$ and **f**, while in (d) the NAND combining element (nd4 in the diagram) is fed with the reverse: **e** and $\overline{\mathbf{f}}$.

2.4 Creating a scalar expression

With a larger number of variables and a limited number of logic element inputs, a single logic element AND may not be sufficient to implement the conjunction. For example, if the maximum number of inputs to a logic element AND is 5, a six-letter conjunction **a b c d e f** would require two logic element IRs:

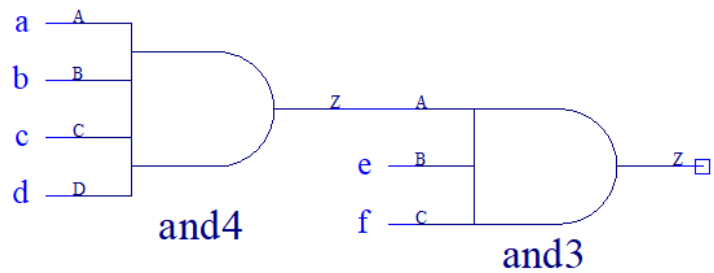


Figure 5: Realisation of the six-letter conjunction a b c d e f

The situation is similar with the realisation of "long" disjunction. In this case, putting identical letters after the parentheses and forming a parenthetical expression often helps.

Let's illustrate this with the following example. Let F = (0, 1, 3, 5, 6, 8, 9, 11, 14, 15, 19, 23, 24, 25, 28, 30, 31, 32, 36, 39, 45, 48, 56, 57, 59, 62). The Karnaugh map for this function looks like this:

Table 7. Karnaugh map of the given function

def \ abc	000	001	011	010	110	111	101	100
000	1	1	1		1	1		1
001	1	1	1			1		
011	1	1		1		1		
010								
110	1	1	1			1		
111		1	1	1				1
101	1						1	
100			1					1

As the number of "1s" in this Karnaugh map is quite large, it is likely that there may be more than one way of grouping them. However, there may be some "1's" in the diagram which are grouped (hereafter we will say "combined") in the only way (the conjunctions corresponding to them will necessarily be included in every minimal function expression). These are the ones to start with.

If you look carefully at the chart, you will find groups of units marked with different colours, which include the following "special" units (defined by the red box).

The group highlighted in yellow will be represented by the conjunction $\mathbf{b\,c\,d\,e\,\overline{f}}$. Let's change its notation to: **-11110.** This notation is also called a **cube**. It is obtained by replacing each letter - variable or its inverse - with a "1" and a "0" respectively, and replacing the omitted letter with a "-". Note that when writing (encoding) conjunctions in this way, it is necessary to keep the order of the variables, in our case **a b c d e f** .

Table 8. Groups of units with "special" "1"

abc

def	000	001	011	010	110	111	101	100
000	1	1	1		1	1		1
001	1	1	1			1		
011	1	1		1		1		
010								
110	1	1	1			1		
111		1	1	1				1
101	1						1	
100			1					1

This representation of conjunctions makes it easier to spot letters common to several conjunctions.

In our diagram we found the following cubes:

- 1 1 1 1 0 (yellow boxes)
0 0 0 - 0 1 (blue boxes)
0 0 - 1 1 0 (greenish boxes)
1 1 1 0 - 1 (grey boxes)
1 0 0 - 0 0 (brown boxes)

Let's replace these cubes (henceforth referred to as "cubes covered") by an "X" in the "1" diagram, to indicate that they no longer need to be taken care of, but can be included in other groups if this gives a better result.

For the remaining "1", we are looking for possible combinations. Looking carefully at the diagram, we find 4 adjacent quadrants of "1" (and "X"):

Table 9. Karnaugh map of the given function after the first combinations

abc

def	000	001	011	010	110	111	101	100
000	1	1	1		1	1		X
001	X	1	1			X		
011	1	1		1		X		
010								
110	X	X	X			X		
111		1	1	1				1
101	X						1	
100			1					X

They correspond to the following cubes:

0 0 - 0 0 -
0 - 1 0 0 -
0 0 - 0 - 1
0 - 1 1 1 -

Replacing the "1" with an "X", we find two pairs of "1s" in the diagram:

Table 10. Karnaugh map of the given function after second combinations

abc

def	000	001	011	010	110	111	101	100
000	X	X	X		1	1		X
001	X	X	X			X		
011	X	X		1		X		
010								
110	X	X	X			X		
111		X	X	1				1
101	X						1	
100			1					X

They correspond to the following cubes:

1 1 - 0 0 0
0 1 0 - 1 1

The remaining two "1's" cannot be combined. They correspond to cubes **1 0 0 1 1 1** and **1 0 1 1 0 1** . So, with minimisation, we got:

- 1 1 1 1 0
0 0 0 - 0 1
0 0 - 1 1 0
1 1 1 0 - 1
1 0 0 - 0 0
0 0 - 0 0 -
0 - 1 0 0 -
0 0 - 0 - 1
0 - 1 1 1 -
1 1 - 0 0 0
0 1 0 - 1 1
1 0 0 1 1 1
1 0 1 1 0 1

Notice that the first two characters of the four cubes are **0 0** :

0 0 0 - 0 1
0 0 - 1 1 0
0 0 - 0 0 -
0 0 - 0 - 1

Hence, the corresponding conjunctions have the letters $\bar{a}\bar{b}$. We can put them before the brackets**:**

$$\bar{a}\bar{b}(\bar{c}e\bar{f} + de\bar{f} + \bar{d}\bar{e} + \bar{d}f)$$

That leaves the cubes:

- 1 1 1 1 0
1 1 1 0 - 1
1 0 0 - 0 0
0 - 1 0 0 -
0 - 1 1 1 -
1 1 - 0 0 0
0 1 0 - 1 1
1 0 0 1 1 1
1 0 1 1 0 1

Notice that the last two cubes (since the conjunction corresponding to each of them has 6 letters and the maximum number of inputs of the logical element AND is 5, it is necessary to raise at least one letter) have even three identical symbols (**1 0 1 1)** :

1 0 0 1 1 1
1 0 1 1 0 1

By taking the common letters of their corresponding conjunctions, we have:

$$a\bar{b}df\ (\bar{c}e + c\bar{e})$$

Among the remaining cubes, the following close cubes can be found:

- 1 1 1 1 0
1 1 1 0 - 1
1 0 0 - 0 0
0 - 1 0 0 -
0 - 1 1 1 -
1 1 - 0 0 0
0 1 0 - 1 1

By taking the common letters of their corresponding conjunctions, we have:

$$bc\ (def + a\bar{d}f) + \bar{a}c\ (\bar{d}\bar{e} + de).$$

Left

1 0 0 - 0 0
1 1 - 0 0 0
0 1 0 - 1 1

After taking the common letters from the first two, we have:

$$a\bar{e}\bar{f}\ (\bar{b}\bar{c} + b\bar{d}).$$

The remaining cube will be realised by the 5-input element IR.

2.5 Implementation of the scheme using multiplexers

The use of a multiplexer to realise a function is based on the **propagation theorem**

$$f(x_1, x_2, \dots, x_n) = x_1 \cdot f(1, x_2, \dots, x_n) + x_1 \cdot \overline{f(0, x_2, \dots, x_n)}.$$

In the case of 4 variables, using the propagation in terms of **x1** and **x2**, we get:

$$f(x_1, x_2, x_3, x_4) = x_1 \cdot f(1, x_2, x_3, x_4) + \overline{x_1} \cdot f(0, x_2, x_3, x_4) =$$

$$\begin{aligned} &= x_1 x_2 \cdot f(1, 1, x_3, x_4) \\ &+ x_1 \overline{x_2} \cdot f(1, 0, x_3, x_4) \\ &+ \overline{x_1} x_2 \cdot f(0, 1, x_3, x_4) \\ &+ \overline{x_1}\, \overline{x_2} \cdot f(0, 0, x_3, x_4) \end{aligned}$$

The design of a scheme using a 4-1 multiplexer (see Figure 6) requires:

- extract address variables from the normal disjunction expression, the number of which depends on the number of address lines in the SD of the selected multiplexer; in this example, we will use ***a*** and ***b*** as address variables;
- rearrange the expression by putting these variables before the brackets.

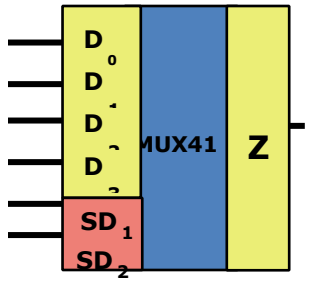


Figure 6. Multiplexer 4-1

For convenience, let's write our function (see Table 4) ***f*** = (**3**, ***4, 5, 6, 7, 11,*** **12**, ***14, 15***) in the form of cubes, with the address variables ***a*** and ***b*** highlighted:

11 Table 1. Function cubes

Minterm number	Cube	Minterm number	Cube
3	0011	11	1011
4	0100	12	1100
5	0101	14	1110
6	0110	15	1111
7	0111		

Putting these variables before the parentheses, the expression looks like this:

$$f = \bar{a}\,\bar{b}\,(c\,d) +$$
$$\bar{a}\,b\,(\bar{c}\,\bar{d} + c\,\bar{d} + c\,d + \bar{c}\,d) +$$
$$a\,\bar{b}\,(c\,d) +$$
$$a\,b\,(\bar{c}\,\bar{d} + c\,d + \bar{c}\,d).$$

Expressions in parentheses can be reduced (minimised). Then we have:

$$f = \bar{a}\,\bar{b}\,(c\,d) +$$
$$\bar{a}\,b\,(1) +$$
$$a\,\bar{b}\,(c\,d) +$$
$$a\,b\,(\bar{d} + c).$$

Another way of transforming the expression to use a multiplexer is based on the analysis of the function's Karnaugh map . Using ***a*** and ***b*** as address variables, the diagram can be split into four parts (they are coloured differently here and the multiplexer input number is indicated at the bottom):

Table 12. Karnaugh map of the function $y = f(a, b, c, d)$

cd \ ab	00	01	11	10
00		1	1	
01		1		
11	1	1	1	1
10		1	1	
	D_0	D_1	D_3	D_2

Minimising the functions of each part gives the same result (above). The circuits implementing these functions are connected to the corresponding inputs of the multiplexer (D_0-D_3).

Notes:

- The complexity of the residual functions depends on the choice of address variables.
- If any of the residual functions is zero or one, the corresponding input of the multiplexer shall be connected to logic zero (vlo) or logic one
  (vhi).

The implementation of the function using a multiplexer is shown in Figure 7.

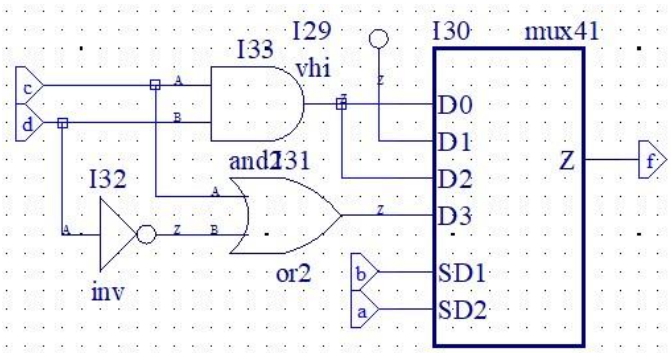


Figure 7: Schematic with multiplexer 4-1

The goodness of the combinatorial scheme can be checked by performing a benchmark test: by providing a sequence of argument combinations to the inputs of the scheme and checking that the outputs of the scheme take the values given in the correctness table.